

Lappeenranta teknillinen yliopisto
School of Business and Management

Software Development Skills

Walid Hassan Tahrim, 003403950

LEARNING DIARY, Software development skills: Mobile MODULE

LEARNING DIARY

Date: 14 Oct 2025

What I did

- **Initial Setup:** Installed the latest Android Studio and created a new **Empty Activity** project using **Jetpack Compose**.
- **App Logic:** Built a functional app with two number fields, an “ADD” button, and a result text display using modern declarative UI.
- **IDE Migration:** Transitioned the development environment from **Android Studio** to **VS Code** because Android Studio was too laggy on my laptop.
- **Version Control:** Organized the GitHub repository into a **COURSEWORK** directory and handled commits for Part 1.
- **Project Maintenance:** Cleaned the project before committing by excluding `.gradle/`, `.idea/`, `build/`, and `local.properties`.
- **Verification:** Verified the UI using Compose Preview (in AS) and successfully ran the app on a physical device (SM-S911B) via VS Code terminal.

What I learned

- **Compose vs XML:** modern templates don’t have `res/layout` by default; UI is defined with `@Composable` functions and `@Preview`.
- **Gradle + Kotlin 2.0:** Compose now requires the **Compose Compiler Gradle plugin** (`org.jetbrains.kotlin.plugin.compose`). The old `kotlinCompilerExtensionVersion` is replaced by the plugin.
- **Dependency management:** using the **Compose BOM** keeps UI libs aligned; typical set includes `activity-compose`, `material3`, `ui-tooling(-preview)`.
- **Preview workflow:** use **Split/Design** → **Refresh** to avoid running full emulator for quick UI checks.

- **Git hygiene:** keep build artifacts out of the repo; structure coursework and project in separate folders to match rubric.

Problems & how I solved them

- **No XML layout found:** realized the project is Compose-first; switched to a Compose implementation instead of chasing `activity_main.xml`.
- **Unresolved references** (KeyboardOptions, Preview): missing Compose deps/imports → added Compose BOM + material3 and the correct `ui.text.input.KeyboardOptions` import.
- **Gradle sync warnings** with Kotlin 2.0: added the **Compose Compiler Gradle plugin** and removed the old compiler extension line.
- **Windows “Path Too Long”** during copy: skipped build/cache files and used a proper `.gitignore`.

Code snippets I wrote (key parts)

- `MainActivity.setContent { MyFirstApp() }`
- Two `OutlinedTextFields` with `KeyboardOptions(keyboardType = KeyboardType.Number)` and a `Button` computing `a + b` with `toIntOrNull()` and showing the result in large `Text`.

Date: 15 Oct 2025

What I did

- **Multi-Activity Architecture:** Built an app with two Activities (MainActivity and SecondActivity) to practice Activity-to-Activity navigation using Intents.
- **Quick Launcher:** Created a launcher screen with three buttons that open external apps (Google Maps, Gmail) and a second activity within the app.
- **Intent Fallback Chains:** Implemented resilient Intent handling with try-catch blocks; if an app is not installed, the app tries to open it via browser, then falls back to Google Play Store.
- **Toast Notifications:** Added user feedback via Toast when apps aren't available.
- **Dynamic Material3 Theming:** Applied Material3 with dynamic colors (Android 12+ support) using `dynamicLightColorScheme()` and `dynamicDarkColorScheme()`.
- **Context Access in Compose:** Used `LocalContext.current` to access the activity context inside Composable functions for launching Intents.
- **Deep Linking:** Used Intent action types (`Intent.ACTION_VIEW`, `Intent.ACTION_SENDTO`) to trigger system intents.

Note: I've done mobile dev before, so I leaned on prior experience with intents and activity lifecycles to move faster once the tooling behaved.

What I learned

- **Activities & Intents (explicit vs implicit):**
 - Explicit intent within the app: `Intent(this, SecondActivity::class.java)`.
 - Implicit intent for external actions: `ACTION_VIEW` with `geo:` or `https:`; `ACTION_SENDTO` with `mailto:`.
- **Package visibility / queries (Android 11+):** When using `resolveActivity` or launching external apps, declaring `<queries>` avoids lint warnings and clarifies which intents I use.
- **Compose Preview vs real device:** Preview is great for layout, **not** for launching external apps. Real testing needs an emulator or phone.

- **Graceful fallbacks:** Always try a primary intent, then browser, then Play Store, and finally a toast—so the button never feels dead.

Problems & how I solved them

- **Endless emulator pain:** boot loops, “Pixel Launcher isn’t responding”, install-write errors, Google splash screen forever.
Fix: gave up on the emulator after many attempts (cold boot, wipe data, new AVD, API 34/35). Switched to my **phone as the emulator**, which worked immediately.
- **No handler apps installed** (on device/emulator): tapping Maps/Gmail did nothing.
Fix: implemented **graceful fallbacks**:
 - Maps: geo: → if fail → https://maps.google.com → if fail → Play Store (Maps) → else toast.
 - Email: mailto: chooser → if fail → Play Store (Gmail) → else Gmail Web → else toast.
- **Lint warnings** (“Use String.toUri”, “Consider adding <queries>”):
Fix: used KTX toUri() and added <queries> for geo, mailto, http/https.

Code snippets I wrote (key parts)

```
// SECOND ACTIVITY (explicit)
startActivity(Intent(this, SecondActivity::class.java))

// MAPS with fallbacks
try {
    startActivity(Intent(Intent.ACTION_VIEW, "geo:0,0?q=".toUri()))
} catch (_: ActivityNotFoundException) {
    try {
        startActivity(Intent(Intent.ACTION_VIEW,
            "https://maps.google.com".toUri()))
    } catch (_: Exception) {
        startActivity(Intent(Intent.ACTION_VIEW,
            "market://details?id=com.google.android.apps.maps".toUri()))
    }
}

// EMAIL with chooser + fallbacks
val mail = Intent(Intent.ACTION_SENDTO, "mailto:".toUri())
startActivity(Intent.createChooser(mail, "Open email app"))
<!-- AndroidManifest.xml: visibility for intents -->
<queries>
    <intent><action android:name="android.intent.action.VIEW"/><data
android:scheme="geo"/></intent>
```

```
<intent><action android:name="android.intent.action.SENDTO"/><data
android:scheme="mailto"/></intent>
<intent><action android:name="android.intent.action.VIEW"/><data
android:scheme="http"/></intent>
<intent><action android:name="android.intent.action.VIEW"/><data
android:scheme="https"/></intent>
</queries>
```

Date: 14 May 2026

What I did

- **LazyColumn List Implementation:** Created a scrollable product list using Compose's LazyColumn instead of the legacy XML ListView approach from the tutorial.
- **Custom List Item Layout:** Built a Card-based item layout with three text fields: product name (24sp, bold), description (14sp, muted), and price (right-aligned, 24sp, bold). Used Row with Spacer for proper spacing and alignment.
- **Data Modeling:** Created an Item data class to represent product data with name, description, price, and drawable resource ID. Populated a list of three items (Peach, Tomato, Squash) with corresponding metadata.
- **Click Navigation:** Implemented OnClick handlers in ListItemCard using Intent to pass the clicked item's index to DetailActivity via intent extras (ITEM_INDEX).
- **Image Loading & Scaling:** Implemented scaledBitmapFromDrawable() function to load drawable resources, convert them to bitmaps, and scale them based on screen width to prevent memory overflow (matching the video's approach).
- **DetailActivity with Image Display:** Built a detail screen that receives the item index, retrieves the corresponding drawable, scales it, and displays it in a Compose Image composable with ContentScale.Crop for proper fitting.

What I learned

- **LazyColumn vs ListView:**
 - LazyColumn is Compose's declarative replacement for ListView
 - Only renders visible items + padding buffer (efficient for large lists)
 - itemsIndexed() composable provides index + item to the lambda
 - verticalArrangement.spacedBy() handles item spacing elegantly
- **Drawable Resource Loading:**
 - activity.getDrawable(drawableId) loads drawable resources as Drawable objects
 - XML drawables (shapes, VectorDrawables) must be converted to Bitmap for image scaling
 - drawableToBitmap() uses Canvas to render drawable → bitmap
 - Bitmap.createScaledBitmap() safely scales images to screen dimensions
- **Image Memory Management:**

- Unscaled images (3000x2000+) cause memory crashes and OutOfMemoryError
- Scaling factor = image width ÷ screen width; divide both dimensions by this ratio
- Using inSampleSize or bitmap scaling before display is essential for large assets
- **Compose Image Composable:**
 - Image(bitmap = ..., contentDescription = ..., contentScale = ContentScale.Crop)
 - asImageBitmap() converts Bitmap to Compose-compatible ImageBitmap
 - ContentScale.Crop maintains aspect ratio while filling the container
- **LaunchedEffect for Side Effects:**
 - Used LaunchedEffect(drawableId) to load images only when the drawable ID changes
 - Avoids reloading images on every recomposition (performance optimization)
 - remember { mutableStateOf() } + LaunchedEffect = proper Compose lifecycle management

Problems & how I solved them

- **App crash on item click:** Initial implementation referenced non-existent bitmap drawables.
 - Fix: Implemented proper drawable → bitmap conversion using Canvas and scaling logic
- **XML shape drawables can't be decoded as bitmaps:** BitmapFactory.decodeResource() returned null for XML-based drawables.
 - Fix: Used activity.getDrawable() + custom drawableToBitmap() conversion
- **Image memory overflow:** Large PNG images caused OutOfMemoryError
 - Fix: Calculated scaling factor from screen width and applied Bitmap.createScaledBitmap()
- **Deprecation warnings for WindowManager.defaultDisplay:** Android 13+ deprecates Display.getWidth()
 - Fix: Code still works but generates warnings; modern approach uses DisplayMetrics (acceptable for coursework)
- **Recomposition performance:** Images were reloading on every screen rotation/recomposition

- Fix: Wrapped image loading logic in LaunchedEffect with drawable ID as key

Code snippets I wrote (key parts)

Item data class:

```
data class Item(  
    val name: String,  
    val description: String,  
    val price: String,  
    val imageResourceId: Int  
)
```

LazyColumn with indexed items:

```
LazyColumn(modifier = Modifier.fillMaxSize(), verticalArrangement =  
Arrangement.spacedBy(8.dp)) {  
  
    itemsIndexed(items) { index, item ->  
  
        ListItemCard(item, index)  
  
    }  
  
}
```

Card-based list item with click handler:

```
Card(modifier = Modifier  
  
    .fillMaxWidth()  
  
    .clickable {  
  
        val intent = Intent(context, DetailActivity::class.java)  
  
        intent.putExtra("ITEM_INDEX", index)  
  
        context.startActivity(intent)  
  
    }  
  
) {
```

```

        Row(modifier = Modifier.fillMaxWidth().padding(16.dp),
horizontalArrangement = Arrangement.SpaceBetween) {

            Column(modifier = Modifier.weight(1f)) {

                Text(text = item.name, fontSize = 24.sp, fontWeight =
FontWeight.Bold)

                Text(text = item.description, fontSize = 14.sp)

            }

            Text(text = item.price, fontSize = 24.sp, fontWeight =
FontWeight.Bold)

        }

    }
}

```

Drawable to Bitmap conversion:

```

fun drawableToBitmap(drawable: Drawable): Bitmap {

    val bitmap = Bitmap.createBitmap(drawable.intrinsicWidth,
drawable.intrinsicHeight, Bitmap.Config.ARGB_8888)

    val canvas = Canvas(bitmap)

    drawable.setBounds(0, 0, canvas.width, canvas.height)

    drawable.draw(canvas)

    return bitmap

}

```

Scaled bitmap loading with LaunchedEffect:

```

var bitmap by remember { mutableStateOf<Bitmap?>(null) }

LaunchedEffect(drawableId) {

    if (drawableId != 0) {

        bitmap = scaledBitmapFromDrawable(activity, drawableId)

    }

}

```

Image display in Compose:

```
if (bitmap != null) {  
  
    Image(  
  
        bitmap = bitmap!!.asImageBitmap(),  
  
        contentDescription = itemName,  
  
        modifier = Modifier.fillMaxWidth().height(400.dp),  
  
        contentScale = ContentScale.Crop  
  
    )  
  
}
```

Date: 14 May 2026

What I did

- **Multi-Activity Architecture:** Implemented three activities (MainActivity, DetailActivity, AddReviewActivity) with Intent-based navigation, passing data between screens using `putExtra()` and retrieving with `getStringExtra()`
- **Jetpack Compose UI Framework:** Built all screens using declarative Compose instead of XML layouts, using Material3 components (AppBar, Card, Button, Slider, Switch, FilterChip) with custom purple (#6A1B9A) and orange (#FF7043) theming
- **List Implementation with Search & Sort:** Created a LazyColumn displaying 10 anime reviews with real-time search filtering (case-insensitive title matching) and two-way sorting options (by rating descending, by title ascending)
- **Form Validation & Input Handling:** Built an add review form with OutlinedTextField inputs for title (required), genre, review text (5 lines max), a Slider for rating (1-5 with visual stars), and a Switch toggle for favorites. Submit button only enabled when title is not blank
- **Image Resource Management:** Loaded drawable poster images (80×120dp thumbnails, 300dp detail view) with emoji fallback (📺) for missing resources. Managed exact filename matching to prevent resource lookup failures
- **Data Model & Sample Data:** Created AnimeReview data class with id, title, rating, review text, posterImageId, isFavorite, and genre fields. Populated with 10 hardcoded anime entries in [SampleData.kt](#)
- **Custom Composable Functions:** Built reusable components including AnimeReviewCard (with poster, title, genre, stars, favorite badge) and ShowStars() (visual star rating display)
- **State Management:** Used remember { mutableStateOf() } for local screen state (search query, sort mode, favorite toggle, form inputs) with proper recomposition handling

What I learned

• Compose vs XML Layouts:

- Compose's declarative syntax is more intuitive than XML layout files
- State management with remember/mutableStateOf ties UI updates directly to data changes
- Column/Row + Modifier chains replace nested LinearLayouts
- Composable functions are reusable UI components (not classes)

• Material3 Theme System:

- TopAppBarDefaults.topAppBarColors() customizes header appearance
- CardDefaults provide elevation, colors, and shape
- ButtonDefaults control container color, text color, and disabled states
- FilterChip and Switch are Material3-specific components (not Material2)

• LazyColumn for Efficient Lists:

- Only renders visible items + buffer (memory efficient for large lists)
- itemsIndexed(items) provides both index and item to lambda
- verticalArrangement.spacedBy() handles spacing elegantly
- contentPadding adds space at bottom for FAB visibility

• Intent-Based Navigation:

- startActivity(Intent) launches new activity
- putExtra("key", value) passes data to launched activity
- getStringExtra("key") retrieves passed data in destination activity
- activity.finish() returns to previous screen

• Form Input Validation:

- title.isNotBlank() checks if user entered text
- Conditional button enabling improves UX (disabled state signals requirement)

- Slider valueRange and steps parameters constrain input
- OutlinedTextField maxLines limits text area size

- **Drawable Resource Handling:**

- R.drawable.{filename} references must match exact filenames in drawable/
- painterResource(id = drawableId) loads drawables in Compose
- Missing resources cause compile errors (not runtime crashes if matched correctly)
- Emoji fallback provides graceful degradation

Problems & how I solved them

- **Material3 Experimental APIs causing compilation errors:** TopAppBar, CardDefaults, ButtonDefaults flagged as experimental

- Fix: Added @OptIn(ExperimentalMaterial3Api::class) annotation before composable functions (Kotlin requirement)

- **Icon imports failing (DarkMode, LightMode icons don't exist):** Tried to add theme toggle button but icons weren't found in Material Icons library

- Fix: Removed theme toggle feature entirely; kept clean header without unnecessary UI

- **Search not updating list:** Initial implementation used read-only variable instead of mutable state

- Fix: Changed searchQuery to mutable state with remember { mutableStateOf() } and made LazyColumn depend on filtered data

- **Sort chips not toggling:** onClick handler wasn't updating sort state properly

- Fix: Used sortBy as mutable state; FilterChip.selected property binds to sortBy == "rating" condition

- **Poster images showing as blank:** R.drawable references weren't matching filenames

- Fix: Added emoji fallback (📺) for posterImageId == 0; verified all 10 anime JPG filenames matched exactly (kuroko_no_basket.jpg, etc.)

Code snippets I wrote (key parts)

AnimeReview data model:

```
data class AnimeReview(  
    val id: String,  
    val title: String,  
    val rating: Int,  
    val review: String,  
    val posterImageId: Int,  
    val isFavorite: Boolean,  
    val genre: String  
)
```

LazyColumn with search and sort:

```
val filtered = mutableListOf<AnimeReview>()  
  
for (review in allReviews) {  
  
    if (review.title.contains(search, ignoreCase = true)) {  
  
        filtered.add(review)  
  
    }  
  
}  
  
val sorted = if (sort == "rating") {  
  
    filtered.sortedByDescending { it.rating }  
  
} else {  
  
    filtered.sortedBy { it.title }  
  
}  
  
LazyColumn(modifier = Modifier.fillMaxSize().padding(horizontal = 8.dp))  
{
```

```

itemsIndexed(sorted) { index, review ->

    AnimeReviewCard(review) {

        val intent = Intent(context, DetailActivity::class.java)

        intent.putExtra("REVIEW_ID", review.id)

        context.startActivity(intent)

    }

}
}

```

AnimeReviewCard with poster and rating:

```

@Composable

fun AnimeReviewCard(review: AnimeReview, onClick: () -> Unit) {

    Card(modifier = Modifier.fillMaxWidth().clickable { onClick() })

    {

        Row(modifier = Modifier.fillMaxWidth().padding(12.dp)) {

            Box(modifier = Modifier.size(80.dp,
120.dp).clip(RoundedCornerShape(8.dp))) {

                if (review.posterImageId != 0) {

                    Image(painter = painterResource(id =
review.posterImageId),

                        contentDescription = review.title, contentScale
= ContentScale.Crop)

                } else {

                    Text("📺", fontSize = 32.sp)

                }

            }

            Column(modifier = Modifier.weight(1f).padding(start = 12.dp))

            {

                Text(review.title, fontWeight = FontWeight.Bold)

                Text(review.genre, fontSize = 12.sp, color = Color.Gray)

```



```

        ShowStars(review.rating)
    }
}
}
}

```

Add Review form with validation:

```

@Composable
fun AddReviewScreen(activity: AddReviewActivity) {
    var titleText by remember { mutableStateOf("") }
    var ratingValue by remember { mutableStateOf(5f) }

    OutlinedTextField(value = titleText, onValueChange = { titleText = it
},

                    label = { Text("Anime Title") }, singleLine = true)

    Slider(value = ratingValue, onValueChange = { ratingValue = it },
           valueRange = 1f..5f, steps = 3)

    Button(onClick = { if (titleText.isNotBlank()) activity.finish() },
           enabled = titleText.isNotBlank()) {
        Text("Add Review")
    }
}

```

Navigation with Intent extras:

```

// In MainActivity - launch detail
val intent = Intent(context, DetailActivity::class.java)

intent.putExtra("REVIEW_ID", review.id)

```

```
context.startActivity(intent)
```

```
// In DetailActivity - receive data
```

```
val reviewId = intent.getStringExtra("REVIEW_ID") ?: ""
```

```
val review = getSampleReviews().find { it.id == reviewId } ?:  
getSampleReviews()[0]
```